

Learn to code — free 3,000-hour curriculum

Continuous Integration (CI)

Imagine you're part of a team of six developers, all working on the same project. Without a proper system, chaos would ensue.

Let's say Mr. A is building a new login feature, Mrs. B is fixing a bug in the search bar, and Mr. C is tweaking the dashboard UI—all at the same time. If everyone is editing the same "folder" or codebase directly, things could go horribly wrong: *"Hey! Who just broke the app?!"*



To keep everything in order, teams use **Version Control Systems (VCS)** like GitHub, GitLab, or BitBucket. Think of it as a digital workspace where everyone can safely collaborate without stepping on each other's toes. 📁 ✨

Here's how Continuous Integration fits into this process step-by-step:

1. The Main Branch: The General Folder ✨

At the heart of every project is the **main branch**—the ultimate source of truth. It contains the stable codebase that powers your live app. It's where every team member contributes their work, but with one important rule: only tested and approved code gets merged here. 🚀

2. Feature Branches: Personal Workspaces 🛠️

When someone like Mr. A wants to work on a new feature, they create a **feature branch**. This branch is essentially a personal copy of the main branch where they can tinker, write code, and test without

Learn to code — free 3,000-hour curriculum

3. Merging Changes: The CI Workflow 🎉

When Mr. A is satisfied with his feature, he doesn't just shove it into the main branch—CI ensures it's done safely:

- **Automated Tests:** Before merging, CI tools automatically run tests on Mr. A's code to check for bugs or errors. Think of it as a bouncer guarding the main branch, ensuring no bad code gets in. 🕵️
- **Build Verification:** The feature branch code is also "built" (converted into a deployable version of the app) to confirm it works as intended.

Once these checks are passed, Mr. A's feature branch is merged into the main branch. This frequent merging of changes is what we call **Continuous Integration**.

Continuous Delivery (CD)

Continuous Delivery (CD) often gets mixed up with Continuous Deployment, and while they share similarities, they serve distinct purposes in the development lifecycle. Let's break it down! 🤖

The Need for a Staging Area 🏠

In the Continuous Integration (CI) process we discussed above, we primarily dealt with **feature branches** and the **main branch**. But directly merging changes from feature branches into the main branch (which powers the live product) can be risky. Why? 🚫

Learn to code — free 3,000-hour curriculum



Think of the staging branch as a “trial run.” Before unleashing changes to real customers, the codebase from feature branches is merged into the staging branch and deployed to a **staging environment**. This environment is an exact replica of the production environment, but it’s used exclusively by the **Quality Assurance (QA) team** for testing.

The QA team takes the role of a “test driver,” running the platform through its paces just as a real user would. They check for usability issues, edge cases, or bugs that automated tests might miss, and provide feedback to developers for fixes. 🚦 If everything passes, the codebase is cleared for deployment to production.

Continuous Delivery in Action

The process of merging changes into the staging branch and deploying them to the **staging environment** is what we call **Continuous Delivery**. 🛠️ It ensures that the application is always in a deployable state, ready for the next step in the pipeline.

Unlike Continuous Deployment (which we’ll discuss later), Continuous Delivery doesn’t automatically push changes to production (live platform). Instead, it pauses to let humans—namely the QA team or stakeholders—decide when to proceed. This adds an extra layer of quality assurance, reducing the chances of errors making it to the live product. 🕵️

Continuous Deployment (CD)

Learn to code — free 3,000-hour curriculum

merging the codebase and deploying it live for end users (the QA testers or the team lead could do this).

Let's explore what makes Continuous Deployment so powerful (and a little scary)! 🤖

The Last Mile of the CI/CD Pipeline 🛣️

Imagine you've gone through the rigorous process of Continuous Integration: teammates have merged their feature branches, automated tests were run, and the codebase was successfully deployed to the staging environment during Continuous Delivery.

Now, you're confident that the application is free of bugs and ready to shine in the production environment—the live version of your platform used by real customers.

In **Continuous Deployment**, this final step of deploying changes to the live environment happens **automatically**. The pipeline triggers whenever specific events occur, such as:

- A **Pull Request (PR)** is merged into the **main branch**.
- A new **release version** is created.
- A **commit** is pushed directly to the production branch (though this is rare for most teams).

Once triggered, the pipeline springs into action, building, testing, and finally deploying the updated codebase to the production environment. 📡

Learn to code — free 3,000-hour curriculum

Integration, Continuous Delivery, and Continuous Deployment

Aspect	Continuous Integration (CI)
Primary Focus	Merging feature branches into the main/general codebase
Automation Level	Automates testing and building processes for feature branches
Testing Scope	Automated tests run on feature branches to ensure code quality
Branch Involved	Feature branches merging into the main/general or staging branch
Environment Target	Ensures integration and testing within a local environment
Key Goal	Prevent integration conflicts and ensure new changes deploy smoothly
Approval Process	No approval needed. Feature branches are tested and merged automatically
Example Trigger	A developer merges a feature branch into the main branch

Now that we've untangled the mysteries of Continuous Integration, Continuous Delivery, and Continuous Deployment, it's time to roll up our sleeves and put theory into practice 😊.

How to Set Up a Node.js Project with a Web Server and Automated Tests ✨

In this hands-on section, we'll build a Node.js web server with automated tests using Jest. From there, we'll create a CI/CD pipeline with GitHub Actions that automates testing for every pull request to

Learn to code — free 3,000-hour curriculum

..... debugging environment, testing and later to the production environment for live use.

Ready to bring your project to life? Let's get started! 🚀 ✨

Step 1: Install Node.js

To get started, you'll need to have **Node.js** installed on your machine. Node.js provides the JavaScript runtime we'll use to create our web server.

1. Visit <https://nodejs.org/en/download/package-manager>
2. Choose your operating system (Windows, macOS, or Linux) and download the installer.
3. Follow the installation instructions to complete the setup.

To verify that Node.js was installed successfully, open your terminal and run `node -v`. This should display the installed version of Node.js

Step 2: Clone the Starter Repository

The next step is to grab the starter code from GitHub. If you don't have Git installed, you can download it at <https://git-scm.com/downloads>. Choose your OS and follow the instructions to install Git. Once you're set, it's time to clone the repository.

Run the following command in your terminal to clone the boilerplate code:

Learn to code — free 3,000-hour curriculum

This will download the project files from the `initial` branch, which contains the starter template for our Node.js web server.

Navigate into the project directory:

```
cd ci-cd-tutorial
```

Step 3: Install Dependencies

Once you're in the project directory, install the required dependencies for the Node.js project. These are the packages that power the application:

```
npm install --force
```

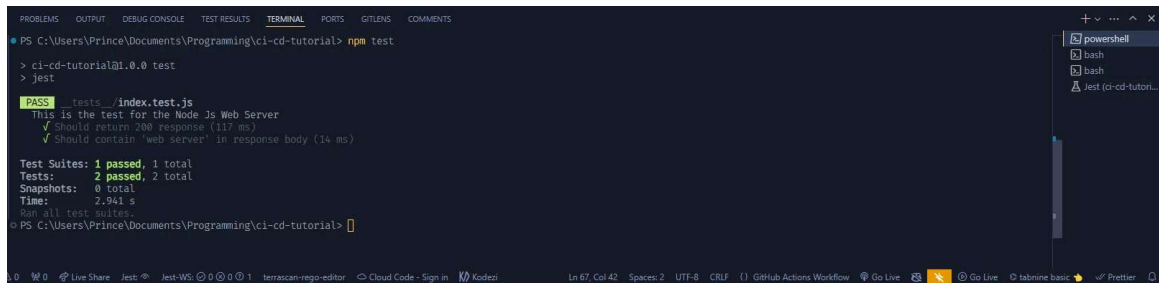
This will download and set up all the libraries specified in the project. Alright, dependencies installed? You're one step closer!

Step 4: Run Automated Tests

Before diving into the code, let's confirm that the automated tests are functioning correctly. Run:

Learn to code — free 3,000-hour curriculum

You should see two successful test results in your terminal. This indicates that the starter project is correctly configured with working automated tests.



```
PS C:\Users\Prince\Documents\Programming\ci-cd-tutorial> npm test
> ci-cd-tutorial@1.0.0 test
> jest

PASS tests ./index.test.js
  This is the test for the Node Js Web Server
    ✓ Should return 200 response (117 ms)
    ✓ Should contain "web server" in response body (14 ms)

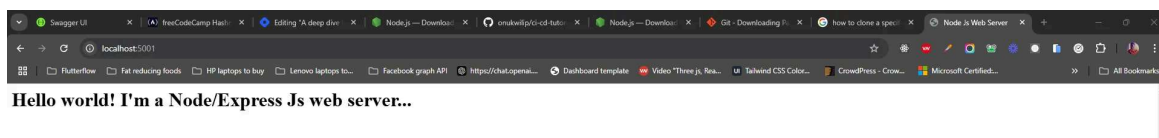
Test Suites: 1 passed, 1 total
Tests: 2 passed, 2 total
Snapshots: 0 total
Time: 2.941 s
Ran all test suites.
```

Step 5: Start the Web Server

Finally, let's start the web server and see it in action. Run the following command:

```
npm start
```

Wait for the application to start running. Open your browser and visit <http://localhost:5000>. 🎉 You should see the starter web server up and running, ready for your CI/CD magic:



Learn to code — free 3,000-hour curriculum

Repository to Host Your Codebase

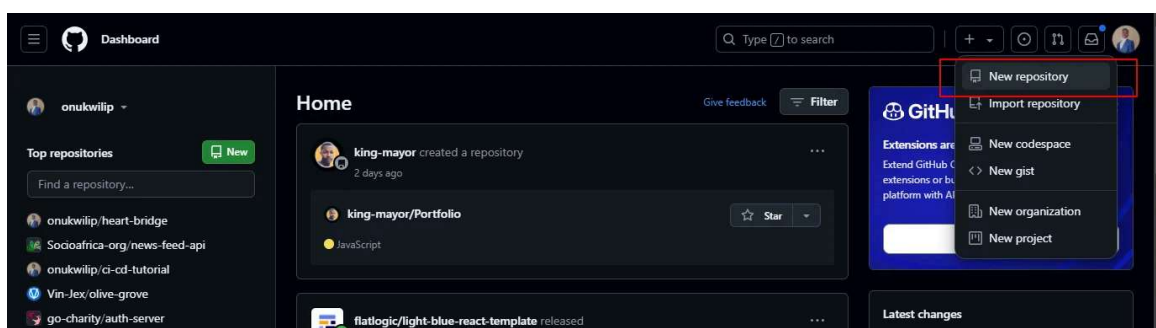


Step 1: Sign In to GitHub

1. **Go to GitHub:** Open your browser and visit GitHub - <https://github.com>.
2. **Sign In:** Click on the **Sign In** button in the top-right corner and enter your username and password to log in, OR create an account if you don't have one by clicking the **Sign up** button.

Step 2: Create a New Repository

Once you're signed in, on the main GitHub page, you'll see a "+" sign in the top-right corner next to your profile picture. Click on it, and select "New repository" from the dropdown.



Now it's time to set the repository details. You'll include:

- **Repository Name:** Choose a name for your repository. For example, you can call it `ci-cd-tutorial`.

Learn to code — free 3,000-hour curriculum

- **Visibility:** Choose whether you want your repository to be **public** (accessible by anyone) or **private** (only accessible by you and those you invite). For the sake of this tutorial, make it **public**.
- **Do Not Check the Add a README File Box: Important:** Make sure you **do not check** the option to **Add a README file**. This will automatically create a `README.md` file in your repository, which could cause conflicts later when you push your local files. We'll add the README file manually if needed later.

After filling out the details, click on **“Create repository”**.

[Donate](#)

Learn to code — free 3,000-hour curriculum

A repository contains all project files, including the revision history. Already have a project repository elsewhere? [Import a repository.](#)

Required fields are marked with an asterisk ().*

Repository template

No template ▾

Start your repository with a template repository's contents.

Owner * **Repository name ***

 onukwilip ▾ /

Great repository names are short and memorable. Need inspiration? How about [literate-waddle](#) ?

Description (optional)

☒  **Public**
Anyone on the internet can see this repository. You choose who can commit.

☐  **Private**
You choose who can see and commit to this repository.

Initialize this repository with:

☐ **Add a README file**
This is where you can write a long description for your project. [Learn more about READMEs.](#)

DO NOT CHECK THIS!

Add .gitignore

.gitignore template: None ▾

Choose which files not to track from a list of templates. [Learn more about ignoring files.](#)

Choose a license

License: None ▾

A license tells others what they can and can't do with your code. [Learn more about licenses.](#)

 You are creating a public repository in your personal account.

Create repository

Step 3: Change the Remote Destination and Push to Your New Repository

Update the Remote Repository URL:

Learn to code — free 3,000-hour curriculum

Copy your repository URL (the URL of the page you were redirected to after creating the repository). It should look similar to this:

`https://github.com/<username>/<repo-name> .`

Open your terminal in the project directory and run the following commands:

```
git remote set-url origin <your-repo-url>
```

Replace `<your-repo-url>` with your GitHub repository URL which you copied earlier.

Rename the Current Branch to `main` :

If your branch is named something other than `main`, you can rename it to `main` using:

```
git branch -M main
```

Push to Your New Repository:

Finally, commit any changes you've made and push your local repository to the new remote GitHub repository by running:

Learn to code — free 3,000-hour curriculum

Now your local codebase is linked to your new GitHub repository, and the files are successfully pushed there. You can verify by visiting your repository on GitHub.

How to Set Up the CI and CD Workflows Within Your Project



Now it's time to create the **CI and CD workflows** for our project! These workflows won't run on your local PC but will be automatically triggered and executed in the cloud once you push your changes to the remote repository. GitHub Actions will detect these workflows and run them based on the triggers you define.

Step 1: Prepare the Workflow Directory

Before adding the CI/CD pipelines, it's a good practice to first create a feature branch. This step mirrors the workflow commonly used in teams, where new features or changes are made in separate branches before they are merged into the main codebase.

To create and switch to a new branch, run the following command:

```
git checkout -b feature/ci-cd-pipeline
```

Learn to code — free 3,000-hour curriculum

Once you finish, you'll be able to merge this feature branch back into `main` or `staging` as part of the pull request process.

In the project's root directory, create a folder named `.github`. Inside `.github`, create another folder called `workflows`.

Any YAML file placed in the `.github/workflows` directory is automatically recognized as a GitHub Actions workflow. These workflows will execute based on specific triggers, such as pull requests, pushes, or releases.

Step 2: Create the Continuous Integration Workflow

We'll now create a CI workflow that automatically tests the application whenever a pull request is made to the `main` or `staging` branches.

First, inside the `workflows` directory, create a file named `ci-pipeline.yml`.

Paste the following code into the file:

```
name: CI Pipeline to staging/production environment
on:
  pull_request:
    branches:
      - staging
      - main
```

Learn to code — free 3,000-hour curriculum

```
env:
  PORT: 5001
steps:
  - name: Checkout
    uses: actions/checkout@v3

  - name: Install dependencies
    run: npm ci

  - name: Test application
    run: npm test

  - name: Build application
    run: |
      echo "Run command to build the application if present"
      npm run build --if-present
```

Explanation of the CI Workflow

Here's a breakdown of each section in the workflow:

1. `name: CI Pipeline to staging/production environment :`
This is the title of your workflow. It helps you identify this pipeline in GitHub Actions.
2. `on :` The `on` parameter is what determines the events that trigger your workflow. When the workflow YAML file is pushed to the remote GitHub repository, GitHub Actions automatically registers the workflow using the configured triggers in the `on` field. These triggers act as event listeners that tell GitHub when to execute the workflow

For example:

If we set `pull_request` as the value for the `on` parameter and specify the branches we want to monitor using the

Learn to code — free 3,000-hour curriculum

```
on:
  pull_request:
    branches:
      - main
      - staging
```

This configuration means that GitHub will trigger the workflow whenever a pull request is made to the `main` or `staging` branches.

Multiple Triggers:

You can define multiple event listeners in the `on` parameter. For instance, in addition to pull requests, you can add a listener for push events.

```
on:
  pull_request:
    branches:
      - main
      - staging
  push:
    branches:
      - main
```

This configuration ensures that the workflow is triggered when:

- A pull request is made to either the `main` or `staging` branch.
- A push is made directly to the `main` branch.

Learn to code — free 3,000-hour curriculum

3. **jobs** : The `jobs` section outlines the specific tasks (or jobs) that the workflow will execute. Each job is an independent unit of work that runs on a separate virtual machine (VM). This isolation ensures a clean, unique environment for every job, avoiding potential conflicts between tasks.

Key Points About Jobs:

1. **Clean VM for Each Job:** When GitHub Actions runs a workflow, it assigns a dedicated VM instance to each job. This means the environment is reset for every job, ensuring there's no overlap or interference between tasks.
2. **Multiple Jobs:** Workflows can have multiple jobs, each responsible for a specific task. For example:
 - A **Test** job to install dependencies and run automated tests.
 - A **Build** job to compile the application.
3. **Job Organization:** Jobs can be organized to run:
 - **Sequentially:** Ensures one job is completed before the next starts, for example the Test job must finish before the Build job. This sequential flow mimics the "pipeline" structure.
 - **Simultaneously:** Multiple jobs can run in parallel to save time, especially if the jobs are independent of one another.

Learn to code — free 3,000-hour curriculum

- Installs dependencies.
- Runs automated tests.
- Builds the application.

 **Learn more about jobs:** Dive into the [GitHub Actions jobs documentation here](#).

4. `runs-on: ubuntu-latest` : Specifies the operating system the job will run on. GitHub provides pre-configured virtual environments, and we're using the latest Ubuntu image.
5. `env` : Sets environment variables for the job. Here, we define the **PORT** variable used by our application.
6. **Steps:** Steps define the individual actions to execute within a job:
 - `Checkout` : Uses the `actions/checkout` action to clone the repository containing the codebase in the feature branch into the virtual machine instance environment. This step ensures the pipeline has access to the project files.
 - `Install dependencies` : Runs `npm ci` to install the required Node.js packages.
 - `Test application` : Runs the automated tests using the `npm test` command. This validates the codebase for errors or failing test cases.
 - `Build application` : Builds the application if a build script is defined in the `package.json`. The `--if-`

Learn to code — free 3,000-hour curriculum

Now that we've completed the CI pipeline, which runs on pull requests to the `main` or `staging` branches, let's move on to setting up the **Continuous Delivery (CD)** and **Continuous Deployment** pipelines. 🚀

Step 3: The Continuous Delivery and Deployment Workflow

First, create the Pipeline File:

In the `.github/workflows` folder, create a new file called `cd-pipeline.yml`. This file will define the workflows for automating delivery and deployment.

Next, paste the configuration:

Copy and paste the following configuration into the `cd-pipeline.yml` file:

```
name: CD Pipeline to Google Cloud Run (staging and production)
on:
  push:
    branches:
      - staging
  workflow_dispatch: {}
  release:
    types: published

env:
  PORT: 5001
  IMAGE: ${vars.IMAGE}:${github.sha}

jobs:
  test:
    runs-on: ubuntu-latest
    name: Setup, test, and build project
    steps:
      - name: Checkout
```

Learn to code — free 3,000-hour curriculum

```
- name: Test application
  run: npm test
build:
  needs: test
  runs-on: ubuntu-latest
  name: Setup project, Authorize GitHub Actions to GCP and Docker Hub,
  steps:
    - name: Checkout
      uses: actions/checkout@v3

    - name: Authenticate for GCP
      id: gcp-auth
      uses: google-github-actions/auth@v0
      with:
        credentials_json: ${{ secrets.GCP_SERVICE_ACCOUNT }}

    - name: Set up Cloud SDK
      uses: google-github-actions/setup-gcloud@v0

    - name: Authenticate for Docker Hub
      id: docker-auth
      env:
        D_USER: ${{secrets.DOCKER_USER}}
        D_PASS: ${{secrets.DOCKER_PASSWORD}}
      run: |
        docker login -u $D_USER -p $D_PASS

    - name: Build and tag Image
      run: |
        docker build -t ${{env.IMAGE}} .

    - name: Push the image to Docker hub
      run: |
        docker push ${{env.IMAGE}}

    - name: Enable the Billing API
      run: |
        gcloud services enable cloudbilling.googleapis.com --project=${{

    - name: Deploy to GCP Run - Production environment (If a new release
      if: github.event_name == 'release' && github.event.action == 'put
      run: |
        gcloud run deploy ${{vars.GCR_PROJECT_NAME}} \
          --region ${{vars.GCR_REGION}} \
          --image ${{env.IMAGE}} \
```

Learn to code — free 3,000-hour curriculum

```
if: github.ref != 'refs/heads/main'
run: |
  echo "Deploying to staging environment"
  # Deploy service with to staging environment
  gcloud run deploy ${vars.GCR_STAGING_PROJECT_NAME} \
    --region ${vars.GCR_REGION} \
    --image ${env.IMAGE} \
    --platform "managed" \
    --allow-unauthenticated \
    --tag staging \
```

The **CD pipeline** configuration combines Continuous Delivery and Continuous Deployment workflows into a single file for simplicity. It builds on the concepts of CI/CD we discussed earlier, automating testing, building, and deploying the application to Google Cloud Run.

Explanation of the CD pipeline:

1. Workflow Triggers (on)

- `push` : Workflow triggers on pushes to the `staging` branch.
- `workflow_dispatch` : Enables manual execution of the workflow via the GitHub Actions interface.
- `release` : Triggers when a new release is published.
Example: When a release is published from the `main` branch, the app deploys to the production environment.

- ### 2. Job 1 – Testing the Codebase:
- The first job in the pipeline, `Test`, ensures the codebase is functional and error-free before proceeding with delivery or deployment

Learn to code — free 3,000-hour curriculum

...gine only, after the build is completed successfully, it prepares the application for deployment and manages the actual deployment process.

Here's what happens:

- **Authorization for GCP and Docker Hub:** The workflow authenticates with both Google Cloud Platform (GCP) and Docker Hub. For GCP, it uses the `google-github-actions/auth@v0` action to handle service account credentials stored as secrets. Similarly, it logs into Docker Hub with stored credentials to enable image uploads.
- **Build and Push Docker Image:** The application is built into a Docker image and tagged with a unique identifier (`${{env.IMAGE}}`). This image is then pushed to Docker Hub, making it accessible for deployment.
- **Deploy to Google Cloud Run:** Based on the event that triggered the workflow, the application is **deployed to either the staging or production environment** in Google Cloud Run. A **push** to the `staging` branch deploys to the staging environment (Continuous Delivery), while a **release** from the `main` branch deploys to production (Continuous Deployment).

To ensure the security and flexibility of our pipeline, we rely on external variables and secrets rather than hardcoding sensitive information directly into the workflow file.

Learn to code — free 3,000-hour curriculum



Instead, we use GitHub's **Secrets** to securely store and access this information. Secrets allow us to define variables that are encrypted and only accessible by our workflows. For example:

- **DockerHub Credentials:** We'll add a Docker username and access token to the repository's secrets. These are essential for authenticating with DockerHub to upload the built Docker images.
- **Google Cloud Service Account Key:** This key will grant the pipeline the necessary permissions to deploy the application on **Google Cloud Run** securely.

We'll set up these variables and secrets incrementally as we proceed, ensuring each step is fully secure and functional. 🎯

Set Up a Docker Hub Repository for the Project's Image and Generate an Access Token for Publishing the Image 📦

Before we dive into the steps, let's quickly go over what we're about to do. In this section, you'll learn how to create a Docker Hub repository, which acts like an online storage space for your application's container image.

Learn to code — free 3,000-hour curriculum

generates a Docker image, which we can then use to create a container. In this step, we'll see how our CI/CD pipeline can use to upload your app's image to Docker Hub. Let's get started! 🚀

Step 1: Sign Up for Docker Hub

Here are the steps to follow to sign up for Docker Hub:

1. **Go to the Docker Hub website:** Open your web browser and visit Docker Hub - <https://hub.docker.com/>.
2. **Create an account:** On the Docker Hub homepage, you'll see a button labelled "**Sign Up**" in the top-right corner. Click on it.
3. **Fill in your details:** You'll be asked to provide a few details like your username, email address, and password. Choose a strong password that you can remember.
4. **Agree to the terms:** You'll need to check a box to agree to Docker's terms of service. After that, click "**Sign Up**" to create your account.
5. **Verify your email:** Docker Hub will send you an email to verify your account. Open that email and click on the verification link to complete your account creation.

Step 2: Sign In to Docker Hub

After verifying your email, go back to Docker Hub, and click on "**Sign In**" at the top right. Then you can use the credentials you just created to log in.

Learn to code — free 3,000-hour curriculum

Now that you have an account, you can create an access token. This token will allow your GitHub Actions workflow to securely sign into Docker Hub and upload Docker images.

Once you're logged into Docker Hub, click on your profile picture (or avatar) in the top right corner. This will open a menu. From the menu, click "Account Settings".

Then in the left-hand menu of your account settings, scroll to the "**Security**" tab. This section is where you manage your tokens and passwords.

Now you'll need to create a new access token. In the Security tab, you'll see a link labelled "**Personal access tokens**" – click on it. Click the button labelled "**Generate new token**".

You'll be asked to give your token a description. You can name it something like "GitHub Actions CI/CD" so that you know what it's for.

After giving it a description, click on the "**Access permissions dropdown**" and select "**Read & Write**", or "**Read, Write, Delete**". Click "**Generate**"

Learn to code — free 3,000-hour curriculum

[Settings](#) / [Personal access tokens](#) / New access token

Create access token

A personal access token is similar to a password except you can have many tokens and revoke access to each one at any time. [Learn more](#)

Access token description

Expiration date: **None**

Optional

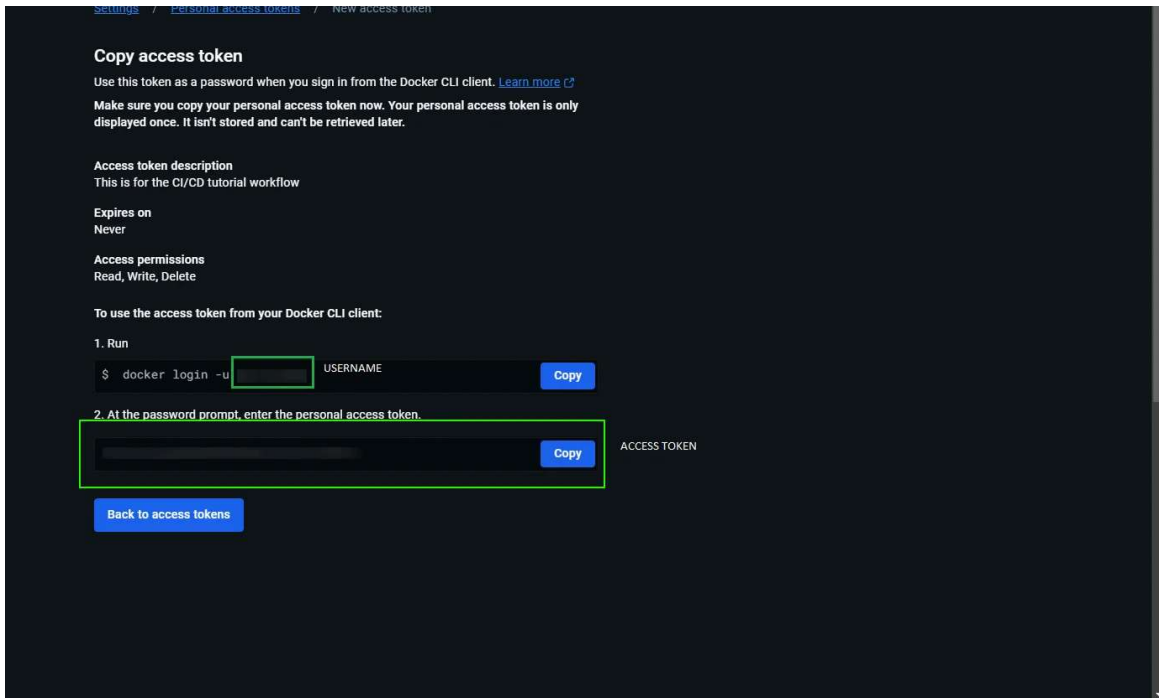
Access permissions: **Public Repo Read-only**

☐ Public Repo Read-only
Read-only

☒ Read & Write
Read, Write, Delete

Now, you need to copy the credentials. After clicking the generate button, Docker Hub will create an access token. **Immediately copy this token along with your username** and save it somewhere safe, like in a file (don't worry, we'll add it to our GitHub secrets). You won't be able to see this token again, so make sure you save it!

Learn to code — free 3,000-hour curriculum



[Settings](#) / [Personal access tokens](#) / New access token

Copy access token

Use this token as a password when you sign in from the Docker CLI client. [Learn more](#)

Make sure you copy your personal access token now. Your personal access token is only displayed once. It isn't stored and can't be retrieved later.

Access token description
This is for the CI/CD tutorial workflow

Expires on
Never

Access permissions
Read, Write, Delete

To use the access token from your Docker CLI client:

1. Run

```
$ docker login -u USERNAME
```

[Copy](#)
2. At the password prompt, enter the personal access token.

[Copy](#) ACCESS TOKEN

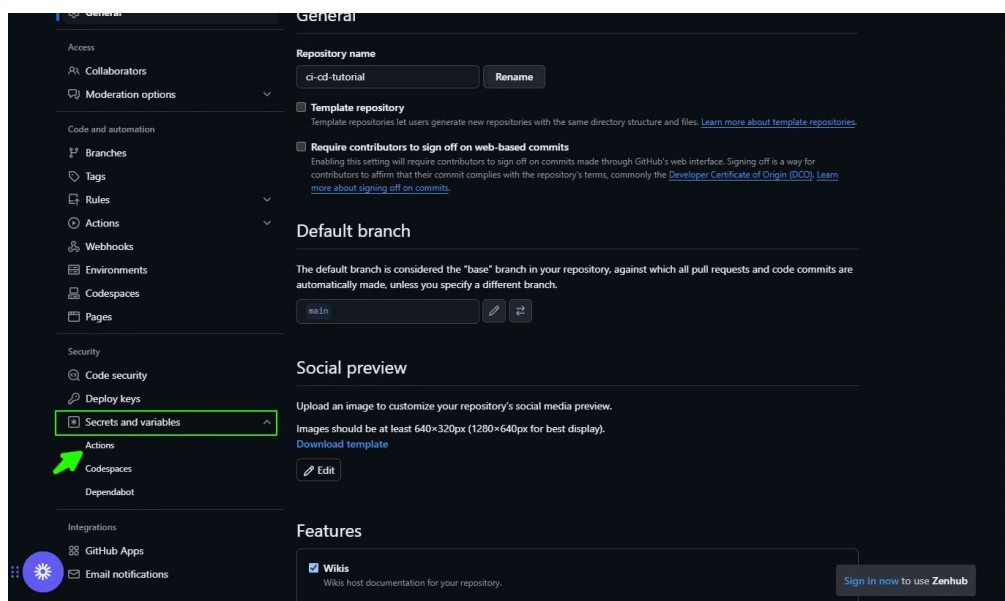
[Back to access tokens](#)

Step 4: Add the Token to GitHub as a Secret

To do this, open your GitHub repository where the codebase is hosted. In the GitHub repo, click on the **Settings** tab (located near the top of your repo page).

Then on the left sidebar, scroll down and click on “**Secrets and Variables**”, then choose “**Actions**”.

Learn to code — free 3,000-hour curriculum



Here are the steps to create and manage your new secret:

1. **Add a new secret:** Click on the “New repository secret” button.
2. **Set up the secret:**
 - In the **Name** field, type `DOCKER_PASSWORD`.
 - In the **Value** field, paste the access token you copied earlier.
3. **Save the secret:** Finally, click **Add secret** to save your Docker access token securely in GitHub.

Then you'll repeat the process for your Docker username. Create a new secret called `DOCKER_USER` and add your Docker username that you copied earlier.

And that's it! Now your CI/CD pipeline can use this token to securely log in to Docker Hub and upload images automatically when triggered.

Learn to code — free 3,000-hour curriculum

Step 5: Creating the Dockerfile for the Project

Before you can build and publish the Docker image to Docker Hub, you need to create a `Dockerfile` that contains the necessary instructions to build your application.

Follow the steps below to create the `Dockerfile` in the root folder of your project:

1. Navigate to your project's root folder.
2. Create a new file named `Dockerfile`.
3. Open the **Dockerfile** in a text editor and paste the following content into it:

```
FROM node:18-slim
```

```
WORKDIR /app
```

```
COPY package.json .
```

```
RUN npm install -f
```

```
COPY . .
```

```
# EXPOSE 5001
```

```
EXPOSE 5001
```

```
CMD ["npm", "start"]
```

Explanation of the Dockerfile:

Learn to code — free 3,000-hour curriculum

- `WORKDIR /app` : Sets the working directory for the application inside the container to `/app`.
- `COPY package.json .` : Copies the `package.json` file into the working directory.
- `RUN npm install -f` : Installs the project dependencies using `npm`.
- `COPY . .` : Copies the rest of the project files into the container.
- `EXPOSE 5001` : This tells Docker to expose port `5001`, which is the port our app will run on inside the container.
- `CMD ["npm", "start"]` : This sets the default command to start the application when the container is run, using `npm start`.

Create a Google Cloud Account, Project, and Billing Account

In this section, we're laying the foundation for deploying our application to Google Cloud. First, we'll set up a Google Cloud account (don't worry, it's free to get started!). Then, we'll create a new project where all the resources for your app will live.

Finally, we'll enable billing so you can unlock the cloud services needed for deployment. Think of this as setting up your workspace in the cloud—organized, ready, and secure! Let's dive in! 

Learn to code — free 3,000-hour curriculum

First, go to [Google Cloud Console](#). If you don't have a Google Cloud account, you'll need to create one.

To do this, click on **Get Started for Free** and follow the steps to set up your account (you'll need to provide payment information, but Google offers \$300 in free credits to get started). If you already have a Google account, simply sign in using your credentials.

Once you've signed in, you'll be taken to your Google Cloud dashboard. This is where you can manage all your cloud projects and resources.

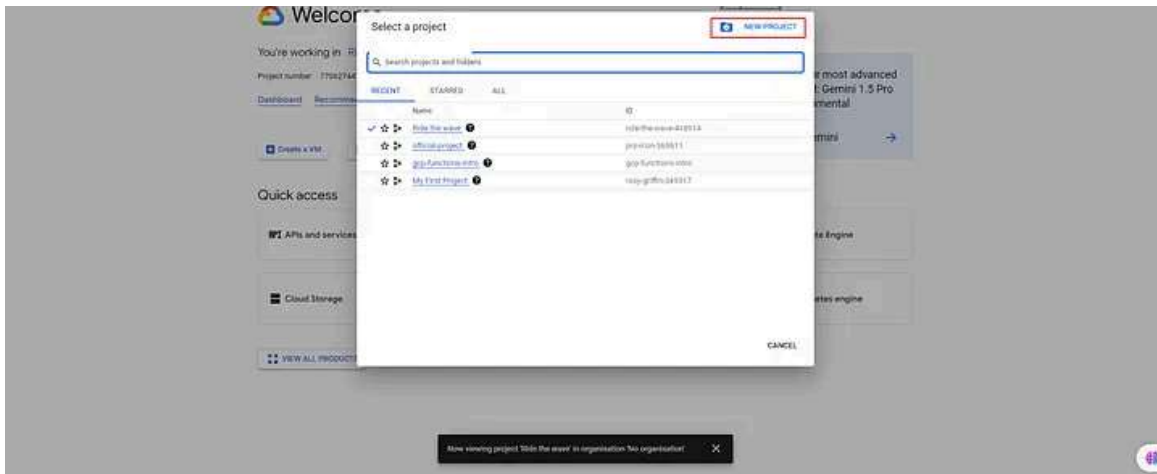
Step 2: Create a New Google Cloud Project



At the top left of the Google Cloud Console, you'll see a drop-down menu beside the Google Cloud logo. Click on this drop-down to display your current projects.

Now it's time to create a new project. In the top-left corner of the pop-up modal, click on the **New Project** button.

Learn to code — free 3,000-hour curriculum



You'll be redirected to a page where you'll need to provide some basic details for your new project. So now enter the following information:

- **Project Name:** Enter a name of your choice for the project (for example, `gcr-ci-cd-project`).
- **Location:** Select a location for your project. You can leave it as the default "No organization" if you're just getting started.

Once you've entered the project name, click the **Create** button. Google Cloud will now start creating your new project. It may take a few seconds.

Step 3: Access Your New Project

After a few seconds, you'll be redirected to your **Google Cloud dashboard**.

Click on the drop-down menu beside the Google Cloud logo again, and you should now see your newly created project listed in the modal where you can select it.

Learn to code — free 3,000-hour curriculum

Step 4: Link A Billing Account To Your Project

To access the billing page, in the Google Cloud Console, find the **Navigation Menu** (the three horizontal lines) at the top left of the screen. Click on it to open a list of options. Scroll down and click on **Billing**. This will take you to the billing section of your Google Cloud account.

[Donate](#)

Learn to code — free 3,000-hour curriculum

[Donate](#)

Learn to code — free 3,000-hour curriculum



Cloud overview



Solutions



PINNED PRODUCTS



API

APIs and services



IAM and admin



Marketplace



Compute Engine



If you haven't set up a billing account yet, you'll be prompted to do so. Click on the "Link billing account" button to start the process.



Kubernetes engine



Now you can create a new billing account (if you don't have one). You'll be redirected to a page where you can either select an existing billing account or create a new one. If you don't already have a billing account, click on "Create a billing account".



Cloud Storage



BigQuery



Provide the necessary details, including:

[Donate](#)

Learn to code — free 3,000-hour curriculum



Cloud Run



SQL



Security



Google Maps Platfor...



Billing



Cloud Run functions

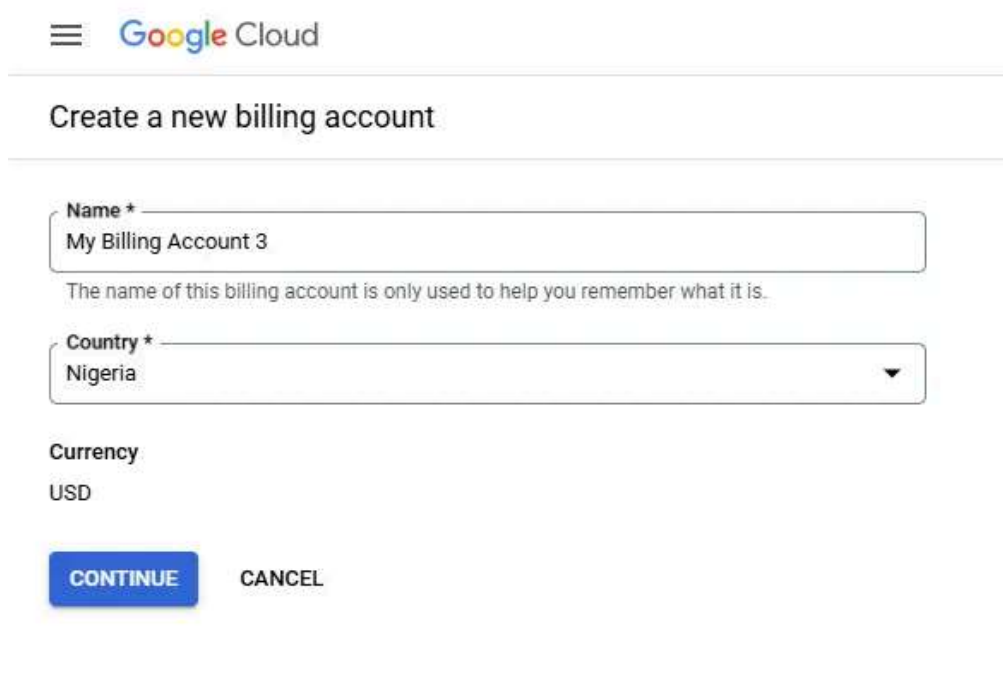


[VIEW ALL PRODUCTS](#)

- **Account name** (for example, "Personal Billing Account" or

Learn to code — free 3,000-hour curriculum
account is based.

- **Currency:** Choose the currency in which you want to be billed.



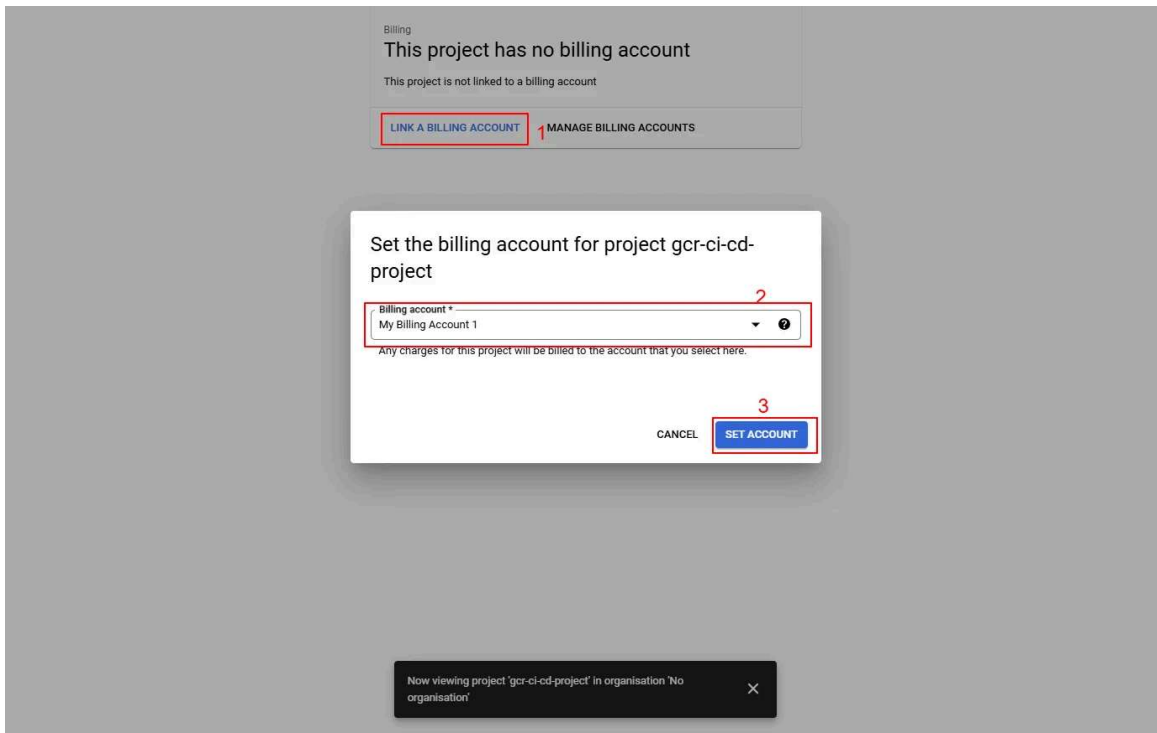
The screenshot shows the Google Cloud 'Create a new billing account' form. At the top, there is a hamburger menu icon and the Google Cloud logo. Below the header, the title 'Create a new billing account' is displayed. The form contains three main sections: 1. 'Name *' with a text input field containing 'My Billing Account 3' and a subtext 'The name of this billing account is only used to help you remember what it is.' 2. 'Country *' with a dropdown menu showing 'Nigeria'. 3. 'Currency' with a dropdown menu showing 'USD'. At the bottom of the form, there are two buttons: a blue 'CONTINUE' button and a grey 'CANCEL' button.

Next, enter your payment information (credit card or bank account details). Google Cloud will verify your payment method, so make sure the information is correct.

Read and agree to the Google Cloud Terms of Service and Billing Account Terms. Once you've done this, click **"Start billing"** to finish setting up your billing account

After setting up your billing account, you'll be taken to a page that asks you to **link** it to your project. Select the billing account you just created or an existing billing account you want to use. Click Set Account to link the billing account to your project.

Learn to code — free 3,000-hour curriculum



After you've linked your billing account to your project, you should see a confirmation message indicating that billing has been successfully enabled for your project.

You can always verify this by returning to the Billing section in the Google Cloud Console, where you'll see your billing account listed.

Create a Google Cloud Service Account to Enable Deployment of the Node.js Application to Google Cloud Run via the CD Pipeline 🚀

Learn to code — free 3,000-hour curriculum

A **service account** allows our CI/CD pipeline to authenticate and interact with Google Cloud services programmatically. By assigning specific roles (permissions), we ensure the service account can only perform tasks related to deployment, such as managing Google Cloud Run.

The **service account key** is a JSON file containing the credentials used for authentication. We securely store this key as a GitHub secret to protect sensitive information.

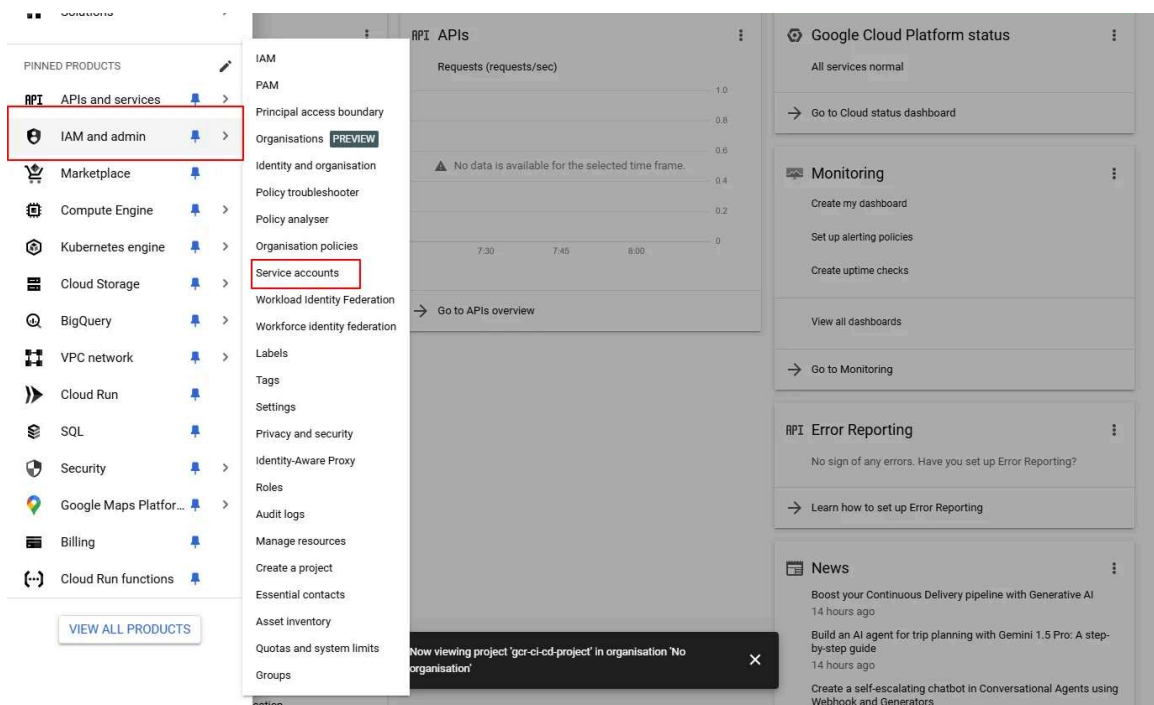
Step 1: Open the Service Accounts Page

Here are the steps you can follow to set up your service account and get your key:

First, visit the Google Cloud Console at <https://console.cloud.google.com/>. Ensure you've selected the correct project (e.g. `gcr-ci-cd-project`). To change projects, click the drop-down menu next to the Google Cloud logo at the top-left corner and select your project.

Then navigate to the Navigation Menu (three horizontal lines in the top-left corner) and click on **IAM & Admin > Service Accounts**.

Learn to code — free 3,000-hour curriculum



Step 2: Create a New Service Account

Click on the "Create Service Account" button. This will open a form where you'll define your service account details.

Next, enter the Service Account details:

- **Name:** Enter a descriptive name (for example, `ci-cd-sa`).
- **ID:** This will auto-fill based on the name.
- **Description:** Add a description to help identify its purpose, such as "Used for deploying Node.js app to Cloud Run."
- Click **Create and Continue** to proceed.

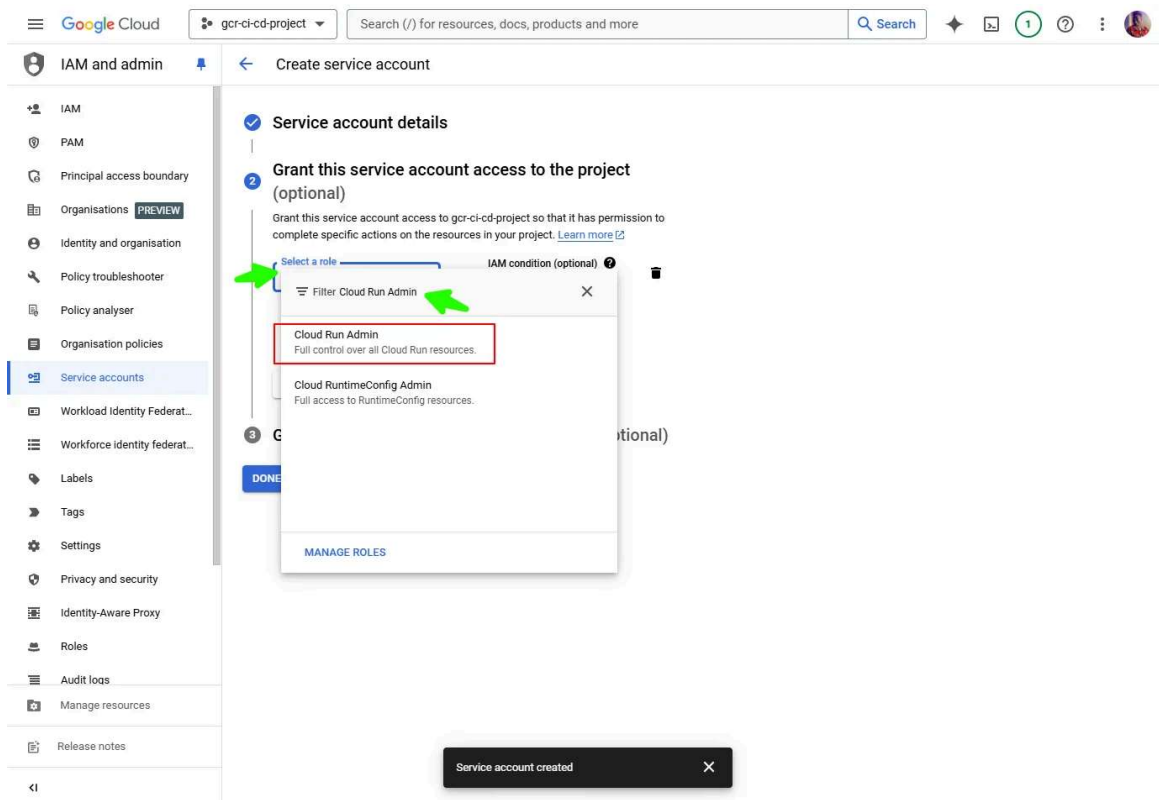
Step 3: Assign Necessary Roles (Permissions)

Learn to code — free 3,000-hour curriculum

- **Cloud Run Admin:** Allows management of Cloud Run services.
- **Service Account User:** Grants the ability to use service accounts.
- **Service Usage Admin:** Enables control over enabling APIs.
- **Viewer:** Provides read-only access to view resources.

To add a role:

- Click on **"Select a Role"**.
- Use the search bar to type the role name (for example, "Cloud Run Admin") and select it.
- Repeat for all four roles.



Learn to code — free 3,000-hour curriculum

The screenshot shows the Google Cloud IAM and admin console. The left sidebar lists various IAM and admin tools, with 'Service accounts' selected. The main panel displays the 'Create service account' wizard. Step 1, 'Service account details', is completed. Step 2, 'Grant this service account access to the project (optional)', is the current step. It shows a list of roles assigned to the service account: 'Cloud Run Admin', 'Service Account User', 'Viewer', and 'Service Usage Admin'. Each role has an 'IAM condition (optional)' link and a trash icon. A '+ ADD ANOTHER ROLE' link is at the bottom. The 'CONTINUE' button is highlighted with a red box. Step 3, 'Grant users access to this service account (optional)', is partially visible at the bottom.

After assigning the roles, click **Continue**.

Step 4: Skip Granting Users Access to the Service Account

On the next screen, you'll see an option to grant additional users access to this service account. Click **Done** to complete the creation process.

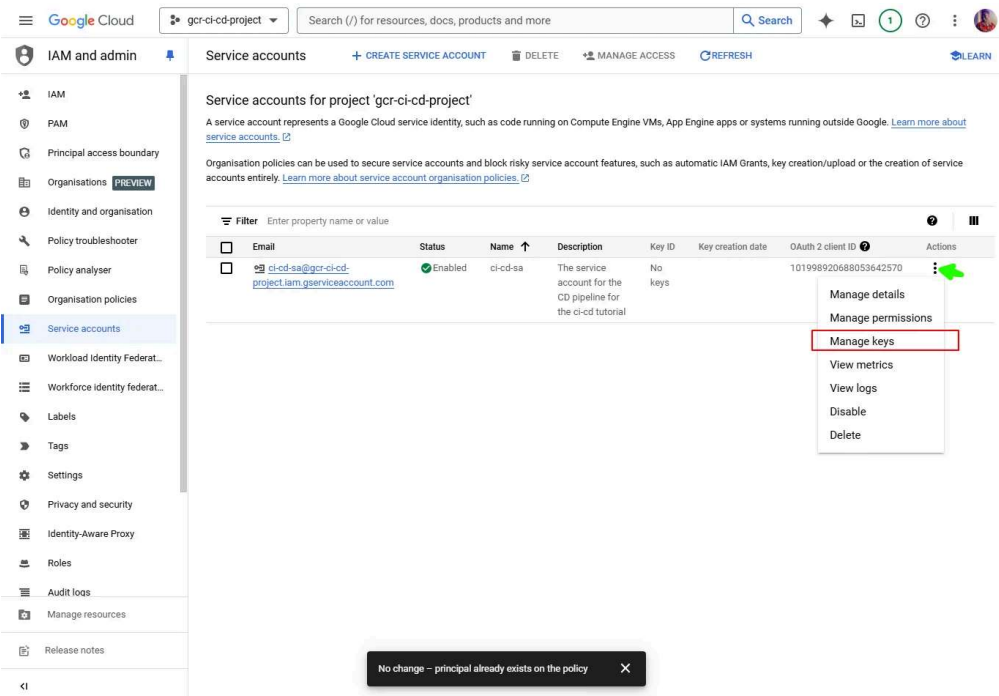
Step 5: Generate a Service Account Key

You should now see your newly created service account in the list. Find the row for your service account (for example, `ci-cd-sa`) and

Learn to code — free 3,000-hour curriculum

To add a new key:

- Click on "Add Key" > "Create New Key".
- In the pop-up dialog, select JSON as the key type.
- Click Create.



Now, download the key file. A JSON file will automatically be downloaded to your computer. This file contains the credentials needed to authenticate with Google Cloud.

Make sure you keep the key secure and store it in a safe location. Don't share it – treat it as sensitive information.

Step 6: Add the Service Account Key to GitHub Secrets 🔒

Learn to code — free 3,000-hour curriculum

Then navigate to the repository you created for this project on GitHub. Click on the **Settings** tab at the top of the repository. Scroll down and find the **Secrets and variables > Actions** section.

Now you need to add a new secret. Click the **"New repository secret"** button. In the **Name** field, enter `GCP_SERVICE_ACCOUNT`. In the **Value** field, paste the JSON content you copied earlier. Click **Add secret** to save it.

Do the same for the `GCP_PROJECT_ID` secret, but now add your Google Project ID as the value. To get your project ID, follow these steps:

1. **Navigate to the Google Cloud Console:** Open Google Cloud Console at <https://console.cloud.google.com/>.
2. **Locate the Project Dropdown:** At the top-left of the screen, next to the **Google Cloud logo**, you will see a drop-down that shows the name of your current project.
3. **View the Project ID:** Click the drop-down, and you'll see a list of all your projects. Your **Project ID** will be displayed next to the project name. It is a unique identifier used by Google Cloud.
4. **Copy the Project ID:** Copy the **Project ID** that is displayed, and add it as the value of the `GCP_PROJECT_ID` secret.

Step 7: Adding External Variables to the GitHub Repository

Learn to code — free 3,000-hour curriculum

Cloud Run services and Docker container registry.

Here are the steps you'll need to follow to do this:

1. First, go to your repository on GitHub.
2. Click the **Settings** tab at the top of the repository. Scroll down to **Secrets and variables > Actions**.
3. Click on the **Variables** tab next to **Secrets**. Click "New repository variable" for each variable. Then you'll need to define these variables:
 - **GCR_PROJECT_NAME** : Set this to the name of your Cloud Run service for the production/live environment. For example, `gcr-ci-cd-app` .
 - **GCR_STAGING_PROJECT_NAME** : Set this to the name of your Cloud Run service for the staging/test environment. For example, `gcr-ci-cd-staging` .
 - **GCR_REGION** : Enter the region where you'd like to deploy the services. For this tutorial, set it to `us-central1` .
 - **IMAGE** : Specify the name of the Docker image/container registry where the published image will be uploaded. For example, `<dockerhub-username>/ci-cd-tutorial-app` .
4. After entering each variable name and value, click **Add variable**.

Learn to code — free 3,000-hour curriculum

To deploy your application, the **Service Usage API** must be enabled in your Google Cloud project. This API allows you to manage Google Cloud services programmatically, including enabling/disabling APIs and monitoring their usage.

Follow these steps to enable it:

1. First, visit the Google Cloud Console at <https://console.cloud.google.com/>.
2. Then make sure you're in the correct project. Click the project drop-down menu near the **Google Cloud logo** at the top-left corner. Select `gcr-ci-cd-project`, or the name you gave your project from the list of projects.
3. Next you'll need to access the API library. Open the **Navigation Menu** (three horizontal lines in the top-left corner). Select **APIs & Services > Library** from the menu.
4. In the API Library, use the search bar to search for "**Service Usage API**".
5. Click on the **Service Usage API** from the search results. On the API's details page, click **Enable**.
6. To verify, go to **APIs & Services > Enabled APIs & Services** in the Google Cloud Console. Confirm that the **Service Usage API** appears in the list of enabled APIs.

Learn to code — free 3,000-hour curriculum

Platform, lists the available...

MANAGE

TRY THIS API



API Enabled

Create the Staging Branch and Merge the Feature Branch into It (Continuous Integration and Continuous Delivery) 🌟

When changes from the `feature/ci-cd-pipeline` branch are merged into the `staging` branch, we complete the **Continuous Integration (CI)** process, and the workflow `ci-pipeline.yml` will run. This ensures that the changes made in the feature branch are tested and integrated into a shared branch.

Once the pull request (PR) is merged into `staging`, the **Continuous Delivery (CD)** pipeline automatically triggers, deploying the application to the staging environment. This simulates how updates are tested in a safe environment before being pushed to production.

Create the `staging` Branch on the Remote Repository

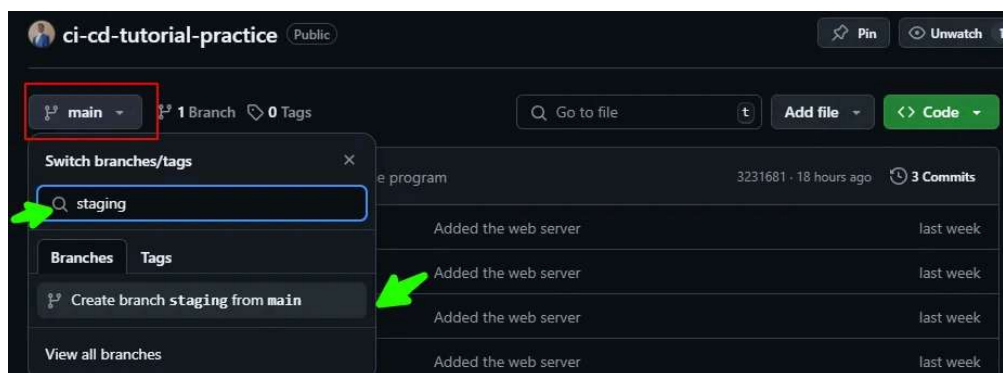
To enable the CI/CD pipeline, we'll first create a `staging` branch on the remote GitHub repository. This branch will serve as the test environment where changes are deployed before they reach the production environment.

To create the `staging` branch directly on GitHub, follow these steps:

Learn to code — free 3,000-hour curriculum

create the new `staging` branch.

2. Then, switch to the `main` branch. On the top of the repository page, locate the **Branch** dropdown (usually labelled as `main` or the current branch name). Click on the dropdown and make sure you are on the `main` branch.
3. Next, create the `staging` branch. In the same dropdown where you see the `main` branch, type `staging` into the text box. Once you start typing, GitHub will offer you the option to create a new branch called `staging`. Select the **Create branch: staging** option from the dropdown.
4. Finally, verify the branch*** After creating the `staging` branch, GitHub will automatically switch to it. You should now see `staging` in the branch dropdown, confirming the new branch was created.



Merge Your Feature Branch into the Staging Branch via a Pull Request (PR)

This process combines both Continuous Integration (CI) and Continuous Delivery (CD). You will commit changes from your feature

Learn to code — free 3,000-hour curriculum

Step 1: Commit Local Changes on Your Feature Branch

First, you'll want to make sure that you are on the correct branch (the feature branch) by running:

```
git status
```

If you are not on the `feature/ci-cd-pipeline` branch, switch to it by running:

```
git checkout feature/ci-cd-pipeline
```

Now, it's time to add your changes you made for the commit:

```
git add .
```

This stages all changes, including new files, modified files, and deleted files.

Next, commit your changes with a clear and descriptive message:

Learn to code — free 3,000-hour curriculum

Then you can verify your commit by running:

```
git log
```

This will display your most recent commits, and you should see the commit message you just added.

Step 2: Push Your Feature Branch Changes to the Remote Repository

After committing your changes, push them to the remote repository:

```
git push origin feature/ci-cd-pipeline
```

This pushes your local changes on the `feature/ci-cd-pipeline` branch to the remote GitHub repository.

Once the push is successful, visit your GitHub repository in a web browser, and confirm that the `feature/ci-cd-pipeline` branch is updated with your new commit.

Step 3: Create a Pull Request to Merge the Feature Branch into Staging

Learn to code — free 3,000-hour curriculum

You should see an alert at the top of the page suggesting you create a pull request for the recently pushed branch (`feature/ci-cd-pipeline`). Click the **Compare & Pull Request** button next to the alert.

Now, it's time to choose the base and compare branches. On the PR creation page, make sure the **base** branch is set to `staging` (this is the branch you want to merge your changes into). The **compare** branch should already be set to `feature/ci-cd-pipeline` (the branch you just pushed). If they're not selected correctly, use the dropdowns to change them.

You'll want to come up with a good PR description for this. Write a clear title and description for the pull request, explaining what changes you're merging and why. For example:

- **Title:** "Merge CI/CD setup changes from feature branch"
- **Description:** "This pull request adds the CI/CD pipelines for GitHub Actions and Docker Hub integration to the project. It includes the configurations for both CI and CD workflows."

Now GitHub will show a list of all the changes that will be merged. Take a moment to review them and ensure everything looks correct.

If all looks good after reviewing, click on the **Create pull request** button. This will create the PR and notify team members (if any) that changes are ready to be reviewed and merged.

Wait a few seconds, and you should see a message indicating that all the checks have passed. Click on the link with the description "CI

[Donate](#)

Learn to code — free 3,000-hour curriculum

Merge CI/CD setup changes from feature branch #1

Open onukwilip wants to merge 1 commit into `staging` from `feature/ci-cd-pipeline`

Conversation 0 Commits 1 Checks 0 Files changed 2

onukwilip commented now

This pull request adds the CI/CD pipelines for GitHub Actions and Docker Hub integration to the project. It includes the configurations for both CI and CD workflows.

Set up CI/CD pipelines for the project ✓ a4e949f

All checks have passed
1 successful check

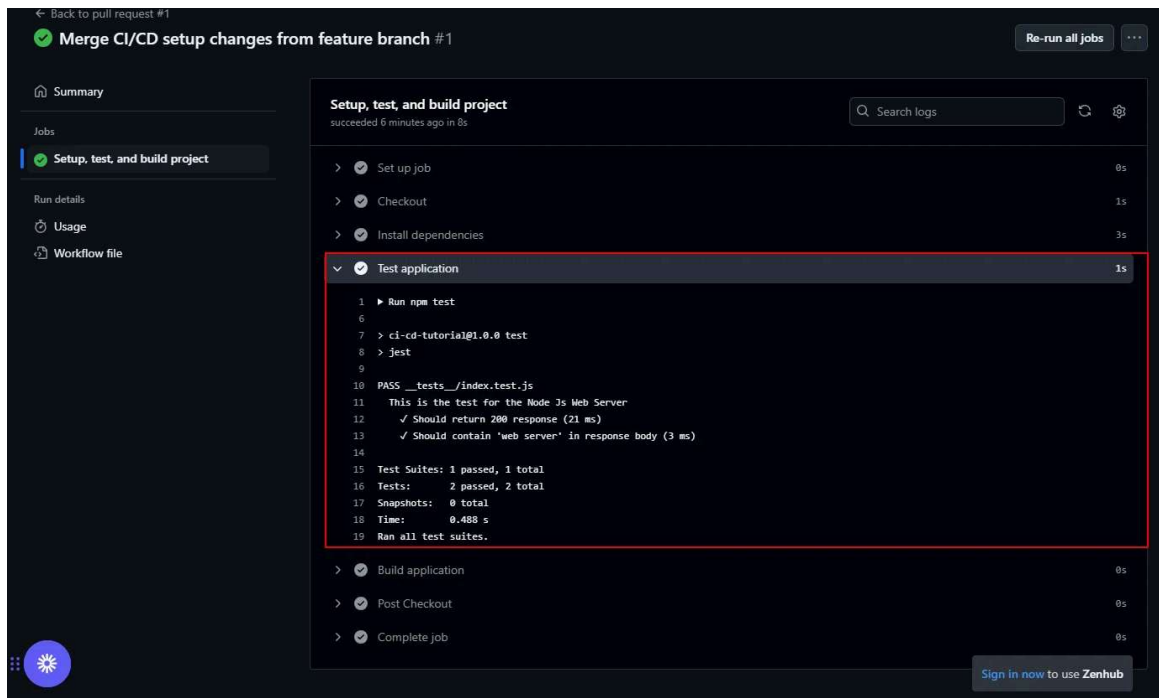
✓ CI Pipeline to staging/production environment / Setup, test, and build project (pull_request) →

No conflicts with base branch
Merging can be performed automatically.

Merge pull request You can also merge this with the command line, view [command line instructions](#).

[Preview](#) Switch back to the classic merge experience · [Give feedback](#)

Learn to code — free 3,000-hour curriculum



The Continuous Integration (CI) Process

The CI process begins when a Pull Request is made to the `staging` branch. It triggers the GitHub Actions workflow defined in the `.github/workflows/ci-pipeline.yml` file. The workflow runs the necessary steps to set up the environment, install dependencies, and build the Node.js application.

It then runs automated tests (using `npm test`) to ensure that the changes do not break any functionality in the codebase. If all these steps are completed successfully, the CI pipeline confirms that the feature branch is stable and ready to be merged into the `staging` branch for further testing and deployment.

Step 4: Merge the Pull Request

Learn to code — free 3,000-hour curriculum

Once the PR has been reviewed and approved, you can merge the PR. To do this, just click on the **Merge pull request** button. Choose **Confirm merge** when prompted.

After merging, you can go to the `staging` branch to verify that the changes were successfully merged.

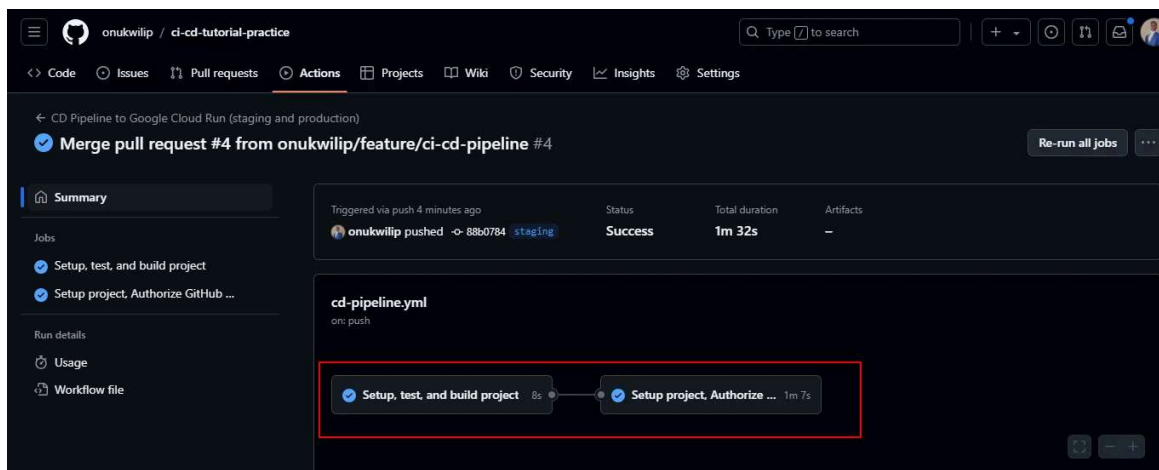
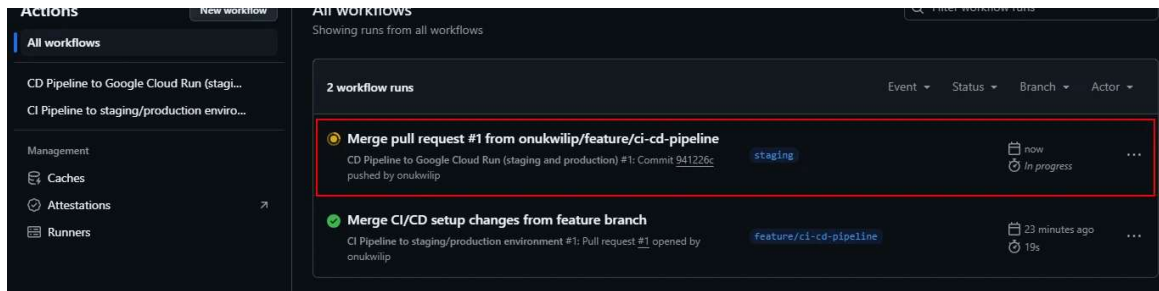
Navigating to the Actions Page After Merging the PR

Once you have successfully merged your pull request from the `feature/ci-cd-pipeline` branch into the `staging` branch, the Continuous Delivery (CD) pipeline will be triggered. To view the progress of the CD pipeline, navigate to the **Actions** tab in your GitHub repository. Here's how to do it:

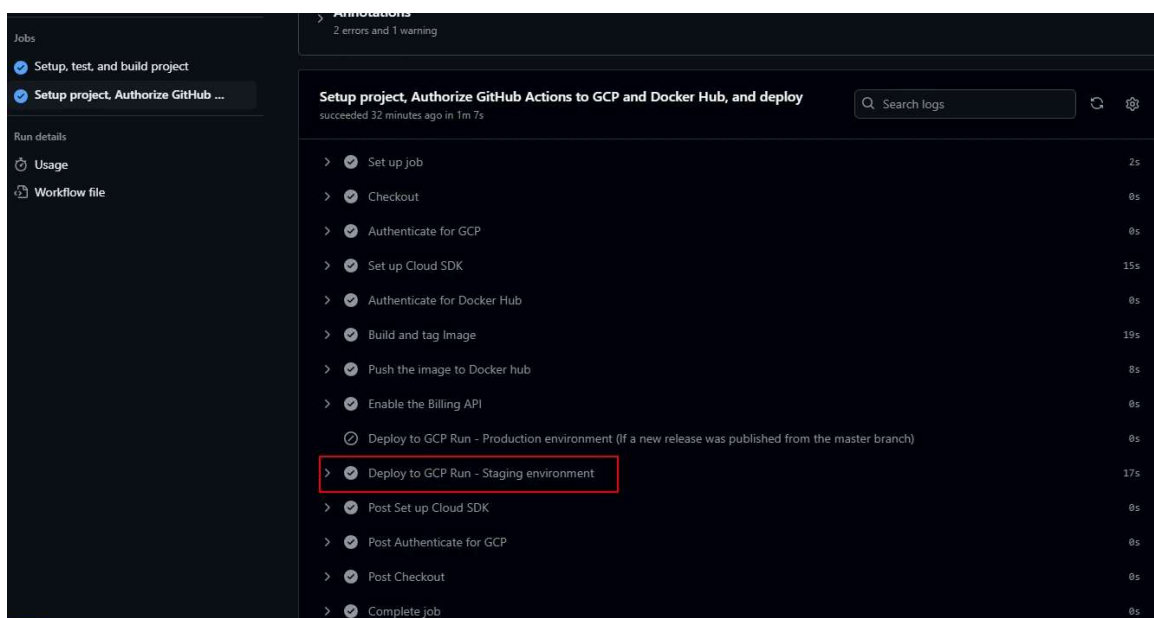
1. Go to your GitHub repository.
2. At the top of the page, you will see the **Actions** tab next to the **Code** tab. Click on it.
3. On the Actions page, you will see a list of workflows that have been triggered. Look for the one labelled **CD Pipeline to Google Cloud Run (staging and production)**. It should appear as a new run after the PR merge.
4. Click on the workflow run to view its progress and see the detailed logs for each step.

[Donate](#)

Learn to code — free 3,000-hour curriculum



Learn to code — free 3,000-hour curriculum



This will allow you to monitor the status of the CD pipeline and check if there are any issues during deployment.

If you look at the CD steps and workflow, you'll see that the step to deploy the application to the **production** environment was skipped, while the step to deploy to the **staging** environment was executed.

Continuous Delivery (CD) pipeline – what's going on:

The **Continuous Delivery (CD) Pipeline** automates the process of deploying the application to Google Cloud Run (testing environment). This workflow is triggered by a push to the `staging` branch, which happens after the changes from the feature branch are merged into `staging`. It can also be manually triggered via `workflow_dispatch` or upon a new release being published.

The pipeline consists of multiple stages:

Learn to code — free 3,000-hour curriculum

pass, the process moves forward.

2. **Build Job:** The next step builds the Docker image of the Node.js application, tags it, and then pushes it to Docker Hub.
3. **Deployment to GCP:** After the image is pushed, the workflow authenticates to Google Cloud and deploys the application. If the event is a release (that is, a push to the `main` branch), the application is deployed to the production environment. If the event is a push to `staging`, the app is deployed to the staging environment.

The CD process ensures that any changes made to the `staging` branch are automatically tested, built, and deployed to the staging environment, ready for further validation. When a release is published, it will trigger deployment to production, ensuring your app is always up to date.

Accessing the Deployed Application in the Staging Environment on Google Cloud Run



Once the deployment to Google Cloud Run is successfully completed, you'll want to access your application running in the **staging** environment. Follow these steps to find and visit your deployed application:

1. Navigate to the Google Cloud Console

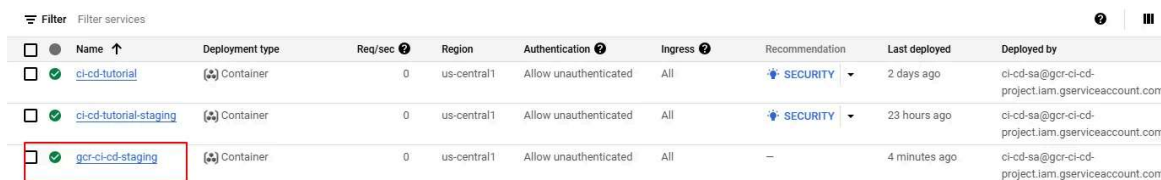
Open the Google Cloud Console in your browser by visiting <https://console.cloud.google.com>. If you're not already signed in, make sure you log in with your Google account.

Learn to code — free 3,000-hour curriculum

In the Google Cloud Console, use the Search bar at the top or navigate through the left-hand menu: Go to **Cloud Run** (you can type this into the search bar, or find it under **Products & services > Compute > Cloud Run**). Click on **Cloud Run** to open the Cloud Run dashboard.

3. Select Your Staging Service

In the **Cloud Run** dashboard, you should see a list of all your services deployed across various environments. Find the service associated with the staging environment. The name should be similar to what you defined in your workflow (for example, `gcr-ci-cd-staging`).



Filter	Filter services								
☐	Name ↑	Deployment type	Req/sec	Region	Authentication	Ingress	Recommendation	Last deployed	Deployed by
☐	ci-cd-tutorial	Container	0	us-central1	Allow unauthenticated	All	SECURITY	2 days ago	ci-cd-sa@gcr-ci-cd-project.iam.gserviceaccount.com
☐	ci-cd-tutorial-staging	Container	0	us-central1	Allow unauthenticated	All	SECURITY	23 hours ago	ci-cd-sa@gcr-ci-cd-project.iam.gserviceaccount.com
☒	gcr-ci-cd-staging	Container	0	us-central1	Allow unauthenticated	All	—	4 minutes ago	ci-cd-sa@gcr-ci-cd-project.iam.gserviceaccount.com

4. Access the Service URL

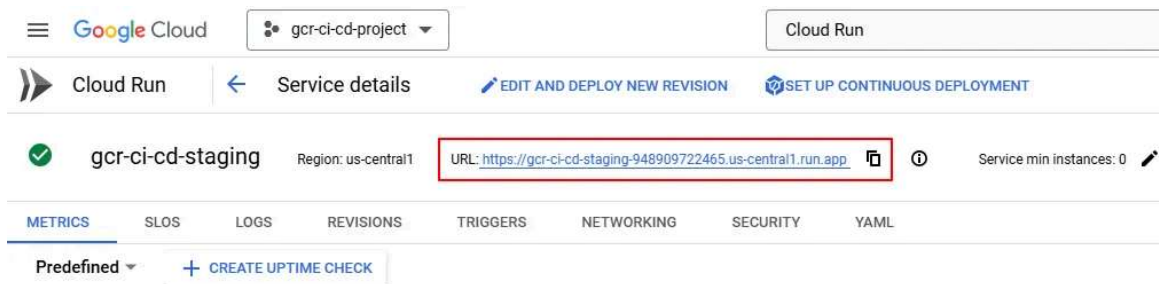
Once you've selected your staging service, you'll be taken to the **Service details** page. This page provides all the important information about your deployed service.

On this page, look for the **URL** section under the **Service URL** heading. The URL will look something like: `https://gcr-ci-cd-staging-
<unique-id>.run.app`.

5. Visit the Application

Click on the **Service URL**, and it will open your staging environment in a new tab in your browser. You can now interact with your application

Learn to code — free 3,000-hour curriculum



Merge the Staging Branch into the Main Branch (Continuous Integration and Continuous Deployment)

In this section, we'll take the updates in the staging branch, merge them into the main branch, and trigger the CI/CD pipeline. This process not only ensures your changes are production-ready but also deploys them to the production/live environment. 🚀

Step 1: Push Local Changes and Open a Pull Request

Why? The first step involves merging the staging branch into the main branch. Just like in the previous Continuous Delivery process, this ensures the integration of thoroughly tested updates.

Here's how to do it:

First, visit the GitHub repository where your project is hosted.

Learn to code — free 3,000-hour curriculum

... explaining why these updates are ready for production deployment.

Step 2: Continuous Integration (CI) Pipeline Execution

After merging the pull request, the **Continuous Integration (CI)** pipeline will automatically execute to validate that the changes are still stable when integrated into the **main branch**.

Pipeline Steps:

- **Code Checkout:** The workflow fetches the latest code from the **main branch**.
- **Dependency Installation:** The pipeline installs all required dependencies.
- **Testing:** Automated tests are run to validate the application's stability.

Step 3: Create a New Release

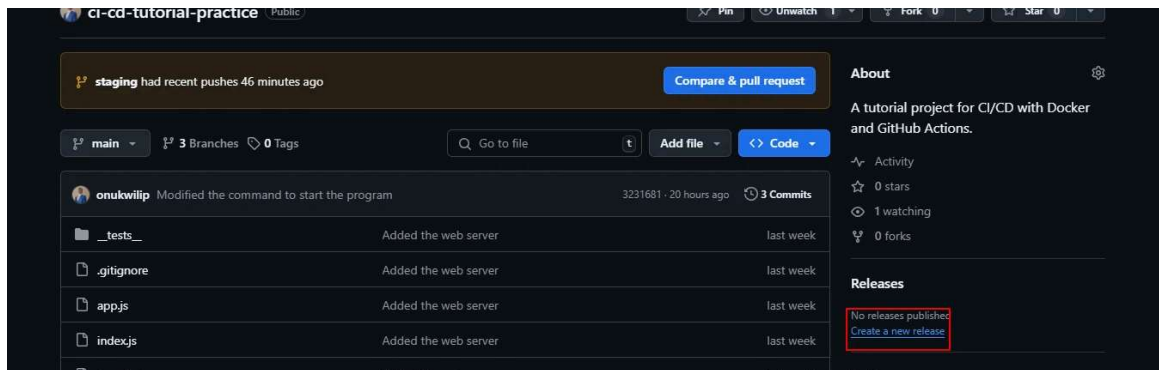
The Continuous Deployment (CD) workflow to deploy to the production environment is triggered by the creation of a new release from the main branch.

Let's walk through the steps to create a release.

On your GitHub repository page, click on the **Releases** section (located under the **Code** tab).

[Donate](#)

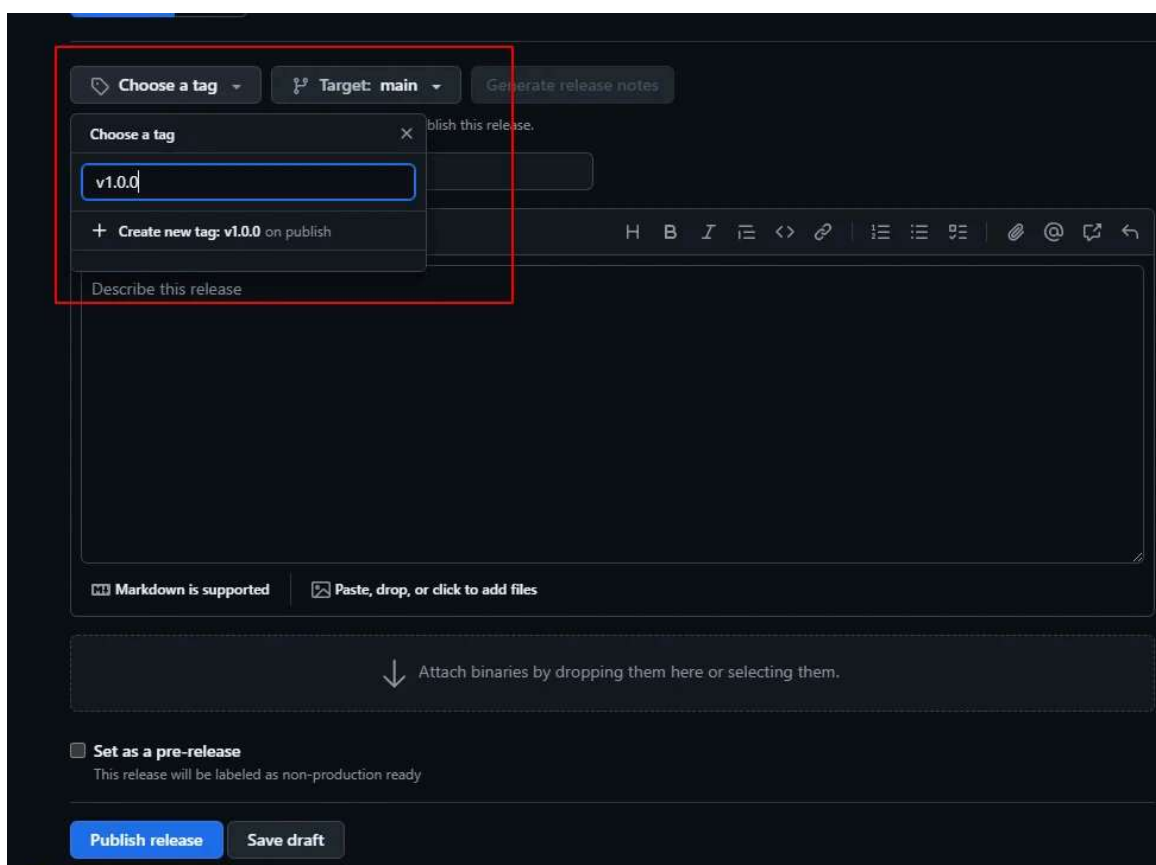
Learn to code — free 3,000-hour curriculum



Next, click **Draft a new release**. Set the **Target** branch to **main**. Enter a **Tag version** (for example, `v1.0.0`) following semantic versioning. Add a **Release title** and an optional description of the changes.

Then, click **Publish Release** to finalize.

Learn to code — free 3,000-hour curriculum



Why run the Continuous Deployment pipeline on release instead of on push? 🤔

In our setup, we decided not to trigger the Continuous Deployment (CD) pipeline every time changes are pushed to the main branch. Instead, we trigger it only when a new release is created. This gives the team more control over when updates are deployed to the production environment.

Imagine a scenario where developers are working on new features—they may push changes to the main branch as part of their regular workflow, but these features might not be complete or ready for users yet. Automatically deploying every push could accidentally expose

Learn to code — free 3,000-hour curriculum

By requiring a release to trigger the deployment, the team gets a chance to finalize and polish all changes before they go live.

For example, developers can test new features in the staging environment, fix any issues, and merge those changes into the main branch without worrying about them immediately appearing in production. This workflow ensures that only well-tested and complete features make their way to your end users.

Ultimately, this approach helps maintain a smooth user experience. Instead of seeing half-built features or unexpected changes, users only see updates that are ready and functional. It also gives the team the flexibility to push changes to the main branch frequently—preventing merge conflicts and making collaboration easier—while keeping control over what gets deployed live. 🚀

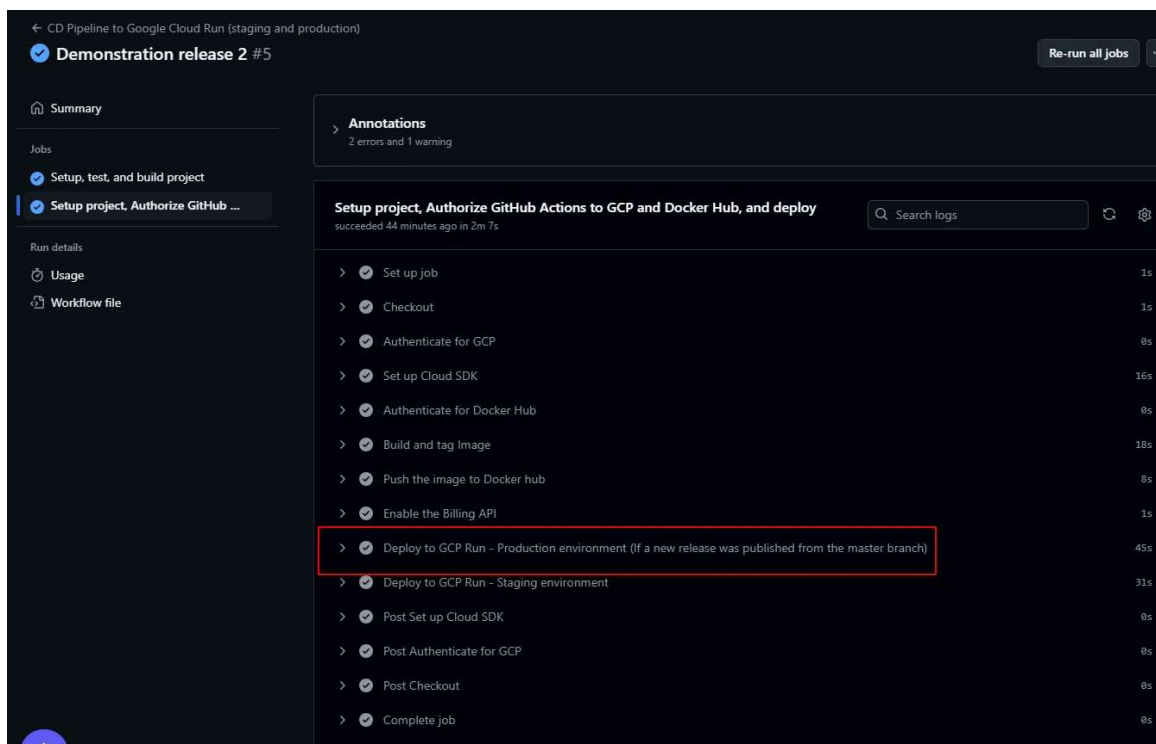
Step 4: Navigate to the Actions Page

After the release is published, the CD pipeline for the production environment is triggered. To monitor this repeat the process taken for the Continuous Delivery workflow, follow these steps:

1. **Go to the GitHub Actions tab:** In your GitHub repository, click on the **Actions** tab.
2. **Locate the deployment workflow:** Look for the **CD Pipeline to Google Cloud Run (staging and production)** workflow. You'll notice that the workflow has been triggered on the **main branch** due to the push event.

Learn to code — free 3,000-hour curriculum

This time, the Continuous delivery workflow deploys the application to the **production/live** environment.



Step 5: Access the Live Application

Once the deployment is complete, go to Google Cloud Console at <https://console.cloud.google.com>.

Navigate to **Cloud Run** from the menu. Select the service corresponding to the **production environment** (for example, `gcr-ci-cd-app`).

Locate the **Service URL** in the service details page. Open the URL in your browser to access the live application.

Learn to code — free 3,000-hour curriculum

Conclusion

In this article, we explored how to build and automate a CI/CD pipeline for a Node.js application, using GitHub Actions, Docker Hub, and Google Cloud Run.

We set up workflows to handle Continuous Integration by testing and integrating code changes and Continuous Delivery to deploy those changes to a staging environment. We also containerized our app using Docker and deployed it seamlessly to Google Cloud Run.

Finally, we implemented Continuous Deployment, ensuring updates to the production environment happen only when a release is created from the main branch.

This approach gives teams the flexibility to push and test incomplete features without impacting end users. By following these steps, you've built a robust pipeline that makes deploying your application smoother, faster, and more reliable.

Study Further

If you would like to learn more about Continuous Integration, Delivery, and Deployment you can check out the courses below:

- [Continuous Integration and Continuous Delivery \(CI/CD\) \(from IBM Coursera\)](#)
- [GitHub Actions - The Complete Guide \(from Udemy\)](#)
- [Learn CI/CD by buliding a project \(freeCodeCamp tutorial\)](#)